

AutoGluon Cross-Validation vs. Holdout on Palmer Penguins (Body Mass Regression)

Goal: Show why cross-validation (CV) with AutoGluon often outperforms a simple train/test holdout for tabular **regression**.

Dataset: Palmer Penguins — predict the numeric target `body_mass_g` from numeric and categorical features.

What you'll learn

- How to load the Penguins dataset.
- How to run **holdout** training in AutoGluon (no CV).
- How to run **CV + stacking/bagging** in AutoGluon and inspect the leaderboard.
- How to compare regression metrics (RMSE, MAE, R^2) on an **unseen test set**.
- Quick residual diagnostics (Predicted vs Actual, Residuals).

Instructor note: Set `TIME_LIMIT_SEC` lower (e.g., 60) for quick demos or higher (e.g., 300) for better accuracy.

1) Setup

If your environment already has the required libraries, you can skip the `pip` cell.

```
In [1]: #!/pip -q install seaborn palmerpenguins  
#!/pip -q install autogluon.tabular
```

```
In [2]: import os, sys, io  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from pathlib import Path  
import shutil # cross-platform directory cleanup  
  
from autogluon.tabular import TabularPredictor  
  
RANDOM_STATE = 42  
np.random.seed(RANDOM_STATE)  
TIME_LIMIT_SEC = 120 # adjust for class time  
TEST_SIZE = 0.2
```

```
C:\Users\biauser\anaconda3\envs\ag312\Lib\site-packages\tqdm\auto.py:21: TqdmWarning:
IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidget
s.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
C:\Users\biauser\anaconda3\envs\ag312\Lib\site-packages\requests\__init__.py:86: Req
uestsDependencyWarning: Unable to find acceptable character detection dependency (ch
ardet or charset_normalizer).
  warnings.warn(
```

2) Load the Penguins dataset

We include basic fallbacks so this works reliably across machines.

```
In [3]: def load_penguins_with_fallbacks():
# 1) Try seaborn bundled
try:
    df = sns.load_dataset('penguins')
    if df is not None and len(df) > 0:
        return df
except Exception as e:
    pass
# 2) Try palmerpenguins package
try:
    from palmerpenguins import load_penguins
    df = load_penguins()
    if df is not None and len(df) > 0:
        return df
except Exception:
    pass
# 3) Try direct CSV (requires internet)
try:
    url = 'https://raw.githubusercontent.com/allisonhorst/palmerpenguins/master
    df = pd.read_csv(url)
    return df
except Exception as e:
    raise RuntimeError('Failed to load Penguins dataset via all fallbacks.')

penguins = load_penguins_with_fallbacks()
print(penguins.shape)
penguins.head()
```

(344, 7)

```
Out[3]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	I
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Fe
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Fe
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Fe

3) Quick EDA and target selection

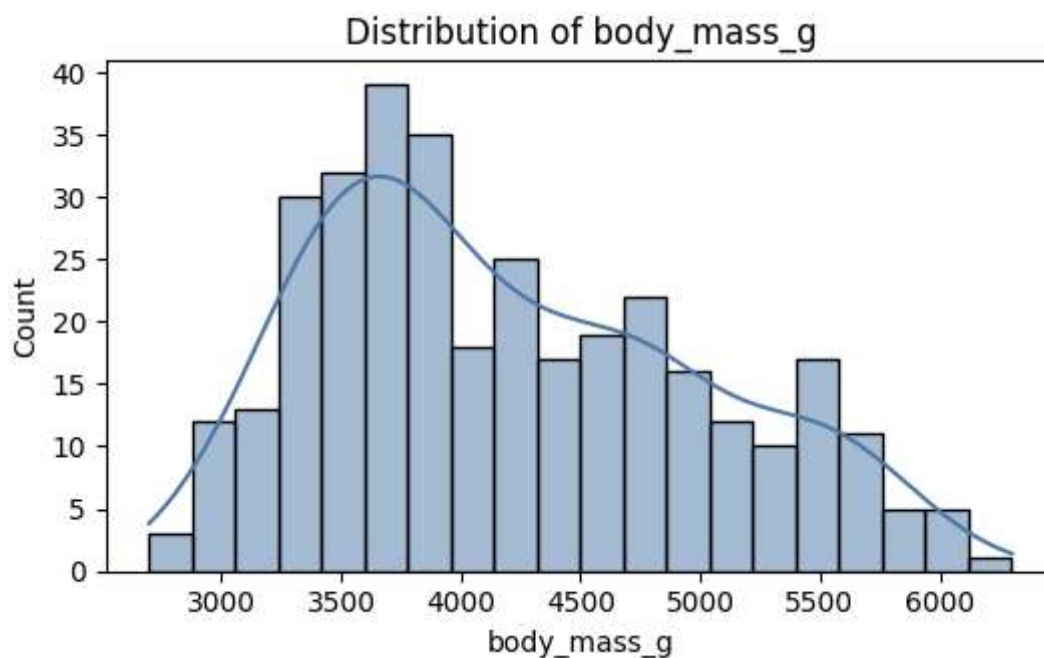
We will predict the numeric target **body_mass_g** (penguin body mass in grams). AutoGluon handles missing values and encoding automatically, so we keep preprocessing minimal.

```
In [4]: label = 'body_mass_g'

# Basic cleaning: drop rows where the label is missing
penguins = penguins.dropna(subset=[label]).copy()

# Quick distribution plot for the target
plt.figure(figsize=(5.5,3.5))
sns.histplot(penguins[label], bins=20, kde=True, color='#4c78a8')
plt.title('Distribution of body_mass_g')
plt.xlabel('body_mass_g')
plt.ylabel('Count')
plt.tight_layout()
plt.show()

# Peek at columns
penguins.info()
```



```
<class 'pandas.core.frame.DataFrame'>
Index: 342 entries, 0 to 343
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                342 non-null    object
1   island                 342 non-null    object
2   bill_length_mm         342 non-null    float64
3   bill_depth_mm          342 non-null    float64
4   flipper_length_mm      342 non-null    float64
5   body_mass_g            342 non-null    float64
6   sex                    333 non-null    object
dtypes: float64(4), object(3)
memory usage: 21.4+ KB
```

```
In [5]: #penguins[penguins['body_mass_g'].isna()]
```

Features and label

We'll include both numeric and categorical columns. AutoGluon can infer types directly from the DataFrame.

```
In [6]: feature_cols = [c for c in penguins.columns if c != label]
penguins[feature_cols + [label]].head()
```

```
Out[6]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	sex	body_ma
0	Adelie	Torgersen	39.1	18.7	181.0	Male	37
1	Adelie	Torgersen	39.5	17.4	186.0	Female	38
2	Adelie	Torgersen	40.3	18.0	195.0	Female	32
4	Adelie	Torgersen	36.7	19.3	193.0	Female	34
5	Adelie	Torgersen	39.3	20.6	190.0	Male	36

4) Create a manual holdout split (no scikit-learn dependency)

We'll randomly sample 20% for the test set using NumPy/pandas. This **test set** is kept untouched during training for final evaluation.

```
In [7]: n = len(penguins)
perm = np.random.permutation(n)
cut = int(n * (1 - TEST_SIZE))
train_idx, test_idx = perm[:cut], perm[cut:]

train_df = penguins.iloc[train_idx].reset_index(drop=True)
test_df = penguins.iloc[test_idx].reset_index(drop=True)
```

```
train_df.shape, test_df.shape
```

```
Out[7]: ((273, 7), (69, 7))
```

```
In [8]: train_df
```

```
Out[8]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
0	Gentoo	Biscoe	46.2	14.5	209.0	4800.0
1	Adelie	Torgersen	37.3	20.5	199.0	3775.0
2	Adelie	Biscoe	39.6	20.7	191.0	3900.0
3	Adelie	Dream	44.1	19.7	196.0	4400.0
4	Adelie	Torgersen	41.5	18.3	195.0	4300.0
...	...	...	...	...	...	...
268	Chinstrap	Dream	45.2	17.8	198.0	3950.0
269	Adelie	Dream	40.8	18.4	195.0	3900.0
270	Gentoo	Biscoe	55.9	17.0	228.0	5600.0
271	Gentoo	Biscoe	50.4	15.3	224.0	5550.0
272	Chinstrap	Dream	49.6	18.2	193.0	3775.0

273 rows × 7 columns



5) AutoGluon with Holdout (no CV / no bagging)

We fit on the training set with default settings (no stacking/bagging). AutoGluon will internally keep a small validation holdout for model selection. Then we evaluate on our **unseen** test set.

```
In [9]: SAVE_PATH_HOLDOUT = Path('agModels-reg-holdout')

# Safe, cross-platform cleanup
if SAVE_PATH_HOLDOUT.exists() and SAVE_PATH_HOLDOUT.is_dir():
    shutil.rmtree(SAVE_PATH_HOLDOUT)

predictor_holdout = TabularPredictor(
    label=label,
    path=str(SAVE_PATH_HOLDOUT),
    problem_type='regression',
    eval_metric='r2' # prioritize R^2 during model selection
).fit(
    train_data=train_df,
```

```
time_limit=TIME_LIMIT_SEC,  
)
```

```

Verbosity: 2 (Standard Logging)
===== System Info =====
AutoGluon Version: 1.5.0
Python Version: 3.12.12
Operating System: Windows
Platform Machine: AMD64
Platform Version: 10.0.26100
CPU Count: 8
Pytorch Version: 2.10.0+cpu
CUDA Version: CUDA is not available
Memory Avail: 6.87 GB / 15.99 GB (43.0%)
Disk Space Avail: 19.81 GB / 127.45 GB (15.5%)
=====
No presets specified! To achieve strong results with AutoGluon, it is recommended to
use the available presets. Defaulting to `medium`...
Recommended Presets (For more details refer to https://auto.gluon.ai/stable/
tutorials/tabular/tabular-essentials.html#presets):
presets='extreme' : New in v1.5: The state-of-the-art for tabular data. Mas
sively better than 'best' on datasets <100000 samples by using new Tabular Foundatio
n Models (TFMs) meta-learned on https://tabarena.ai: TabPFNv2, TabICL, Mitra, TabDP
T, and TabM. Requires a GPU and `pip install autogluon.tabular[tabarena]` to install
TabPFN, TabICL, and TabDPT.
presets='best' : Maximize accuracy. Recommended for most users. Use in c
ompetitions and benchmarks.
presets='best_v150': New in v1.5: Better quality than 'best' and 5x+ faster
to train. Give it a try!
presets='high' : Strong accuracy with fast inference speed.
presets='high_v150': New in v1.5: Better quality than 'high' and 5x+ faster
to train. Give it a try!
presets='good' : Good accuracy with very fast inference speed.
presets='medium' : Fast training time, ideal for initial prototyping.
Using hyperparameters preset: hyperparameters='default'
Beginning AutoGluon training ... Time limit = 120s
AutoGluon will save models to "C:\data\GitHub\BZIA_VM\Autogluon\agModels-reg-holdou
t"
Train Data Rows: 273
Train Data Columns: 6
Label Column: body_mass_g
Problem Type: regression
Preprocessing data ...
Using Feature Generators to preprocess the data ...
Fitting AutoMLPipelineFeatureGenerator...
Available Memory: 7043.92 MB
Train Data (Original) Memory Usage: 0.05 MB (0.0% of available memory)
Inferring data type of each feature based on column values. Set feature_meta
data_in to manually specify special dtypes of the features.
Stage 1 Generators:
Fitting AsTypeFeatureGenerator...
Stage 2 Generators:
Fitting FillNaFeatureGenerator...
Stage 3 Generators:
Fitting IdentityFeatureGenerator...
Fitting CategoryFeatureGenerator...
Fitting CategoryMemoryMinimizeFeatureGenerator...
Stage 4 Generators:
Fitting DropUniqueFeatureGenerator...

```

```

Stage 5 Generators:
  Fitting DropDuplicatesFeatureGenerator...
Types of features in original data (raw dtype, special dtypes):
  ('float', []) : 3 | ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm']
  ('object', []) : 3 | ['species', 'island', 'sex']
Types of features in processed data (raw dtype, special dtypes):
  ('category', []) : 3 | ['species', 'island', 'sex']
  ('float', []) : 3 | ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm']
0.1s = Fit runtime
6 features in original data used to generate 6 features in processed data.
Train Data (Processed) Memory Usage: 0.01 MB (0.0% of available memory)
Data preprocessing and feature engineering runtime = 0.07s ...
AutoGluon will gauge predictive performance using evaluation metric: 'r2'
  To change this, specify the eval_metric parameter of Predictor()
Automatically generating train/validation split with holdout_frac=0.2, Train Rows: 218, Val Rows: 55
User-specified model hyperparameters to be fit:
{
  'NN_TORCH': [{}],
  'GBM': [{'extra_trees': True, 'ag_args': {'name_suffix': 'XT'}}, {}, {'learning_rate': 0.03, 'num_leaves': 128, 'feature_fraction': 0.9, 'min_data_in_leaf': 3, 'ag_args': {'name_suffix': 'Large', 'priority': 0, 'hyperparameter_tune_kwargs': None}}],
  'CAT': [{}],
  'XGB': [{}],
  'FASTAI': [{}],
  'RF': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'squared_error', 'ag_args': {'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}, {'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'squared_error', 'ag_args': {'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
}
Fitting 9 L1 models, fit_strategy="sequential" ...
Fitting model: LightGBMXT ... Training model for up to 119.93s of the 119.93s of remaining time.
  Fitting with cpus=4, gpus=0, mem=0.0/7.0 GB
  0.866 = Validation score (r2)
  2.04s = Training runtime
  0.01s = Validation runtime
Fitting model: LightGBM ... Training model for up to 117.87s of the 117.87s of remaining time.
  Fitting with cpus=4, gpus=0, mem=0.0/7.0 GB
  0.8519 = Validation score (r2)
  0.5s = Training runtime
  0.01s = Validation runtime
Fitting model: RandomForestMSE ... Training model for up to 117.35s of the 117.34s of remaining time.
  Fitting with cpus=8, gpus=0
  0.8525 = Validation score (r2)
  0.74s = Training runtime
  0.07s = Validation runtime

```

```

Fitting model: CatBoost ... Training model for up to 116.50s of the 116.50s of remaining time.
    Fitting with cpus=4, gpus=0
    0.8563 = Validation score (r2)
    8.17s = Training runtime
    0.0s = Validation runtime
Fitting model: ExtraTreesMSE ... Training model for up to 108.32s of the 108.32s of remaining time.
    Fitting with cpus=8, gpus=0
    0.8571 = Validation score (r2)
    0.6s = Training runtime
    0.05s = Validation runtime
Fitting model: NeuralNetFastAI ... Training model for up to 107.63s of the 107.63s of remaining time.
    Fitting with cpus=4, gpus=0, mem=0.0/6.9 GB
    0.8648 = Validation score (r2)
    2.55s = Training runtime
    0.02s = Validation runtime
Fitting model: XGBoost ... Training model for up to 105.04s of the 105.04s of remaining time.
    Fitting with cpus=4, gpus=0
    0.8485 = Validation score (r2)
    0.48s = Training runtime
    0.01s = Validation runtime
Fitting model: NeuralNetTorch ... Training model for up to 104.54s of the 104.53s of remaining time.
    Fitting with cpus=4, gpus=0, mem=0.0/6.9 GB
C:\Users\biauser\anaconda3\envs\ag312\Lib\site-packages\sklearn\compose\_column_transformer.py:975: FutureWarning: The parameter `force_int_remainder_cols` is deprecated and will be removed in 1.9. It has no effect. Leave it to its default value to avoid this warning.
    warnings.warn(
    0.8849 = Validation score (r2)
    2.83s = Training runtime
    0.01s = Validation runtime
Fitting model: LightGBMLarge ... Training model for up to 101.69s of the 101.69s of remaining time.
    Fitting with cpus=4, gpus=0, mem=0.1/6.8 GB
    0.854 = Validation score (r2)
    1.06s = Training runtime
    0.01s = Validation runtime
Fitting model: WeightedEnsemble_L2 ... Training model for up to 119.93s of the 100.41s of remaining time.
    Fitting 1 model on all data | Fitting with cpus=8, gpus=0, mem=0.0/6.8 GB
    Ensemble Weights: {'NeuralNetTorch': 0.818, 'XGBoost': 0.182}
    0.8868 = Validation score (r2)
    0.07s = Training runtime
    0.0s = Validation runtime
AutoGluon training complete, total runtime = 19.7s ... Best model: WeightedEnsemble_L2 | Estimated inference throughput: 3050.8 rows/s (55 batch size)
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("C:\data\GitHub\BZIA_VM\Autogluon\agModels-reg-holdout")

```

```

In [10]: print('Holdout → Test evaluation:')
         holdout_test_metrics = predictor_holdout.evaluate(test_df)

```

```
print('Holdout Leaderboard on Test:')
holdout_lb_test = predictor_holdout.leaderboard(test_df, silent=True)
holdout_lb_test.head()
```

Holdout → Test evaluation:

Holdout Leaderboard on Test:

Out[10]:

	model	score_test	score_val	eval_metric	pred_time_test	pred_time_val	
0	ExtraTreesMSE	0.837562	0.857112	r2	0.099669	0.054199	0
1	NeuralNetTorch	0.832802	0.884872	r2	0.014048	0.008514	2
2	LightGBMXT	0.832754	0.866011	r2	0.035610	0.006516	2
3	WeightedEnsemble_L2	0.831856	0.886807	r2	0.035120	0.018028	3
4	CatBoost	0.829802	0.856349	r2	0.017041	0.004000	8

6) AutoGluon with Cross-Validation + Stacking/Bagging

Enable CV by turning on bagging/stacking. We'll use **5 folds** and **1 stack level**. AutoGluon trains models on folds, blends them, and often gains stability and accuracy.

```
In [11]: SAVE_PATH_CV = Path('agModels-reg-cv')

# Clean up old directory if it exists
if SAVE_PATH_CV.exists() and SAVE_PATH_CV.is_dir():
    shutil.rmtree(SAVE_PATH_CV)

predictor_cv = TabularPredictor(
    label=label,
    path=str(SAVE_PATH_CV),
    problem_type='regression',
    eval_metric='r2'
).fit(
    train_data=train_df,
    time_limit=TIME_LIMIT_SEC,
    auto_stack=True,          # enables bagging + stacking
    num_bag_folds=5,
    num_stack_levels=1,
)
```

```

Verbosity: 2 (Standard Logging)
===== System Info =====
AutoGluon Version: 1.5.0
Python Version: 3.12.12
Operating System: Windows
Platform Machine: AMD64
Platform Version: 10.0.26100
CPU Count: 8
Pytorch Version: 2.10.0+cpu
CUDA Version: CUDA is not available
Memory Avail: 6.72 GB / 15.99 GB (42.0%)
Disk Space Avail: 19.80 GB / 127.45 GB (15.5%)
=====
No presets specified! To achieve strong results with AutoGluon, it is recommended to
use the available presets. Defaulting to `medium`...
Recommended Presets (For more details refer to https://auto.gluon.ai/stable/
tutorials/tabular/tabular-essentials.html#presets):
presets='extreme' : New in v1.5: The state-of-the-art for tabular data. Mas
sively better than 'best' on datasets <100000 samples by using new Tabular Foundatio
n Models (TFMs) meta-learned on https://tabarena.ai: TabPFNv2, TabICL, Mitra, TabDP
T, and TabM. Requires a GPU and `pip install autogluon.tabular[tbarena]` to install
TabPFN, TabICL, and TabDPT.
presets='best' : Maximize accuracy. Recommended for most users. Use in c
ompetitions and benchmarks.
presets='best_v150': New in v1.5: Better quality than 'best' and 5x+ faster
to train. Give it a try!
presets='high' : Strong accuracy with fast inference speed.
presets='high_v150': New in v1.5: Better quality than 'high' and 5x+ faster
to train. Give it a try!
presets='good' : Good accuracy with very fast inference speed.
presets='medium' : Fast training time, ideal for initial prototyping.
Using hyperparameters preset: hyperparameters='default'
Stack configuration (auto_stack=True): num_stack_levels=1, num_bag_folds=5, num_bag_
sets=1
Beginning AutoGluon training ... Time limit = 120s
AutoGluon will save models to "C:\data\GitHub\BZIA_VM\Autogluon\agModels-reg-cv"
Train Data Rows: 273
Train Data Columns: 6
Label Column: body_mass_g
Problem Type: regression
Preprocessing data ...
Using Feature Generators to preprocess the data ...
Fitting AutoMLPipelineFeatureGenerator...
Available Memory: 6980.55 MB
Train Data (Original) Memory Usage: 0.05 MB (0.0% of available memory)
Inferring data type of each feature based on column values. Set feature_meta
data_in to manually specify special dtypes of the features.
Stage 1 Generators:
Fitting AsTypeFeatureGenerator...
Stage 2 Generators:
Fitting FillNaFeatureGenerator...
Stage 3 Generators:
Fitting IdentityFeatureGenerator...
Fitting CategoryFeatureGenerator...
Fitting CategoryMemoryMinimizeFeatureGenerator...
Stage 4 Generators:

```

```

Fitting DropUniqueFeatureGenerator...
Stage 5 Generators:
Fitting DropDuplicatesFeatureGenerator...
Types of features in original data (raw dtype, special dtypes):
('float', []) : 3 | ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm']
('object', []) : 3 | ['species', 'island', 'sex']
Types of features in processed data (raw dtype, special dtypes):
('category', []) : 3 | ['species', 'island', 'sex']
('float', []) : 3 | ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm']
0.1s = Fit runtime
6 features in original data used to generate 6 features in processed data.
Train Data (Processed) Memory Usage: 0.01 MB (0.0% of available memory)
Data preprocessing and feature engineering runtime = 0.07s ...
AutoGluon will gauge predictive performance using evaluation metric: 'r2'
To change this, specify the eval_metric parameter of Predictor()
User-specified model hyperparameters to be fit:
{
  'NN_TORCH': [{}],
  'GBM': [{'extra_trees': True, 'ag_args': {'name_suffix': 'XT'}}, {}, {'learning_rate': 0.03, 'num_leaves': 128, 'feature_fraction': 0.9, 'min_data_in_leaf': 3, 'ag_args': {'name_suffix': 'Large', 'priority': 0, 'hyperparameter_tune_kwargs': None}}],
  'CAT': [{}],
  'XGB': [{}],
  'FASTAI': [{}],
  'RF': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'squared_error', 'ag_args': {'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
  'XT': [{'criterion': 'gini', 'ag_args': {'name_suffix': 'Gini', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'entropy', 'ag_args': {'name_suffix': 'Entr', 'problem_types': ['binary', 'multiclass']}}, {'criterion': 'squared_error', 'ag_args': {'name_suffix': 'MSE', 'problem_types': ['regression', 'quantile']}}],
}
AutoGluon will fit 2 stack levels (L1 to L2) ...
Fitting 9 L1 models, fit_strategy="sequential" ...
Fitting model: LightGBMXT_BAG_L1 ... Training model for up to 79.94s of the 119.93s of remaining time.
2026-03-10 10:21:35,410 INFO util.py:154 -- Missing packages: ['ipywidgets']. Run `pip install -U ipywidgets`, then restart the notebook server for rich notebook output.

Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting Strategy (5 workers, per: cpus=1, gpus=0, memory=0.23%)
C:\Users\biauser\anaconda3\envs\ag312\Lib\site-packages\ray\_private\worker.py:2062: FutureWarning: Tip: In future versions of Ray, Ray will no longer override accelerator visible devices env var if num_gpus=0 or num_gpus=None (default). To enable this behavior and turn off this error message, set RAY_ACCEL_ENV_VAR_OVERRIDE_ON_ZERO=0
  warnings.warn(
0.8817 = Validation score (r2)
1.73s = Training runtime
0.03s = Validation runtime
Fitting model: LightGBM_BAG_L1 ... Training model for up to 64.84s of the 104.83s of remaining time.
Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting

```

```

Strategy (5 workers, per: cpus=1, gpus=0, memory=0.30%)
    0.8759 = Validation score (r2)
    2.04s  = Training runtime
    0.02s  = Validation runtime
Fitting model: RandomForestMSE_BAG_L1 ... Training model for up to 58.27s of the 98.
27s of remaining time.
    Fitting 1 model on all data (use_child_oof=True) | Fitting with cpus=8, gpus
=0
    0.8577 = Validation score (r2)
    0.64s  = Training runtime
    0.11s  = Validation runtime
Fitting model: CatBoost_BAG_L1 ... Training model for up to 57.47s of the 97.47s of
remaining time.
    Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting
Strategy (5 workers, per: cpus=1, gpus=0, memory=6.52%)
    0.8739 = Validation score (r2)
    16.74s = Training runtime
    0.02s  = Validation runtime
Fitting model: ExtraTreesMSE_BAG_L1 ... Training model for up to 36.79s of the 76.79
s of remaining time.
    Fitting 1 model on all data (use_child_oof=True) | Fitting with cpus=8, gpus
=0
    0.8653 = Validation score (r2)
    0.6s   = Training runtime
    0.11s  = Validation runtime
Fitting model: NeuralNetFastAI_BAG_L1 ... Training model for up to 36.04s of the 76.
04s of remaining time.
    Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting
Strategy (5 workers, per: cpus=1, gpus=0, memory=0.00%)
    0.8746 = Validation score (r2)
    12.05s = Training runtime
    0.09s  = Validation runtime
Fitting model: XGBoost_BAG_L1 ... Training model for up to 19.67s of the 59.67s of r
emaining time.
    Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting
Strategy (5 workers, per: cpus=1, gpus=0, memory=0.11%)
    0.8505 = Validation score (r2)
    1.81s  = Training runtime
    0.05s  = Validation runtime
Fitting model: NeuralNetTorch_BAG_L1 ... Training model for up to 12.30s of the 52.2
9s of remaining time.
    Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting
Strategy (5 workers, per: cpus=1, gpus=0, memory=0.00%)
    0.8859 = Validation score (r2)
    11.81s = Training runtime
    0.04s  = Validation runtime
Fitting model: WeightedEnsemble_L2 ... Training model for up to 119.93s of the 35.41
s of remaining time.
    Fitting 1 model on all data | Fitting with cpus=8, gpus=0, mem=0.0/5.0 GB
    Ensemble Weights: {'NeuralNetTorch_BAG_L1': 0.579, 'LightGBMXT_BAG_L1': 0.26
3, 'NeuralNetFastAI_BAG_L1': 0.105, 'XGBoost_BAG_L1': 0.053}
    0.888  = Validation score (r2)
    0.1s   = Training runtime
    0.0s   = Validation runtime
Fitting 9 L2 models, fit_strategy="sequential" ...
Fitting model: LightGBMXT_BAG_L2 ... Training model for up to 35.27s of the 35.26s o

```

```

f remaining time.
    Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting
Strategy (5 workers, per: cpus=1, gpus=0, memory=0.34%)
    0.8746 = Validation score (r2)
    2.06s  = Training runtime
    0.03s  = Validation runtime
Fitting model: LightGBM_BAG_L2 ... Training model for up to 28.50s of the 28.49s of
remaining time.
    Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting
Strategy (5 workers, per: cpus=1, gpus=0, memory=0.30%)
    0.8666 = Validation score (r2)
    2.08s  = Training runtime
    0.02s  = Validation runtime
Fitting model: RandomForestMSE_BAG_L2 ... Training model for up to 21.71s of the 21.
70s of remaining time.
    Fitting 1 model on all data (use_child_oof=True) | Fitting with cpus=8, gpus
=0
    0.8664 = Validation score (r2)
    0.89s  = Training runtime
    0.18s  = Validation runtime
Fitting model: CatBoost_BAG_L2 ... Training model for up to 20.59s of the 20.58s of
remaining time.
    Fitting 5 child models (S1F1 - S1F5) | Fitting with ParallelLocalFoldFitting
Strategy (5 workers, per: cpus=1, gpus=0, memory=6.27%)
    0.8723 = Validation score (r2)
    16.75s = Training runtime
    0.03s  = Validation runtime
Fitting model: WeightedEnsemble_L3 ... Training model for up to 119.93s of the -0.30
s of remaining time.
    Fitting 1 model on all data | Fitting with cpus=8, gpus=0, mem=0.0/6.1 GB
    Ensemble Weights: {'NeuralNetTorch_BAG_L1': 0.579, 'LightGBMXT_BAG_L1': 0.26
3, 'NeuralNetFastAI_BAG_L1': 0.105, 'XGBoost_BAG_L1': 0.053}
    0.888  = Validation score (r2)
    0.07s  = Training runtime
    0.0s   = Validation runtime
AutoGluon training complete, total runtime = 120.4s ... Best model: WeightedEnsemble
_L2 | Estimated inference throughput: 265.7 rows/s (55 batch size)
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("C:\data\Git
Hub\BZIA_VM\Autogluon\agModels-reg-cv")

```

```

In [12]: print('CV → Test evaluation:')
cv_test_metrics = predictor_cv.evaluate(test_df)

print('CV Leaderboard on Test:')
cv_lb_test = predictor_cv.leaderboard(test_df, silent=True)
cv_lb_test.head()

```

CV → Test evaluation:
CV Leaderboard on Test:

Out[12]:

	model	score_test	score_val	eval_metric	pred_time_test	pred_time_val
0	RandomForestMSE_BAG_L1	0.842463	0.857723	r2	0.156459	0.11483
1	RandomForestMSE_BAG_L2	0.842416	0.866416	r2	0.820696	0.63481
2	ExtraTreesMSE_BAG_L1	0.842374	0.865325	r2	0.133190	0.10814
3	CatBoost_BAG_L1	0.840308	0.873950	r2	0.105778	0.02105
4	WeightedEnsemble_L2	0.839921	0.887963	r2	0.285357	0.20818

7) Compare results (unseen test set)

We line up key regression metrics from both approaches. (Exact numbers vary with randomness and time budget.)

```
In [13]: def extract_metric(metrics_dict, key_candidates):
    for k in key_candidates:
        if k in metrics_dict:
            return metrics_dict[k]
    return np.nan

rows = []
rows.append({
    'approach': 'Holdout (no CV)',
    'rmse': extract_metric(holdout_test_metrics, ['root_mean_squared_error', 'rmse']),
    'mae': extract_metric(holdout_test_metrics, ['mean_absolute_error', 'mae']),
    'r2': extract_metric(holdout_test_metrics, ['r2', 'r2_score']),
})
rows.append({
    'approach': 'CV + stacking (5 folds)',
    'rmse': extract_metric(cv_test_metrics, ['root_mean_squared_error', 'rmse']),
    'mae': extract_metric(cv_test_metrics, ['mean_absolute_error', 'mae']),
    'r2': extract_metric(cv_test_metrics, ['r2', 'r2_score']),
})

cmp_df = pd.DataFrame(rows)
cmp_df
```

Out[13]:

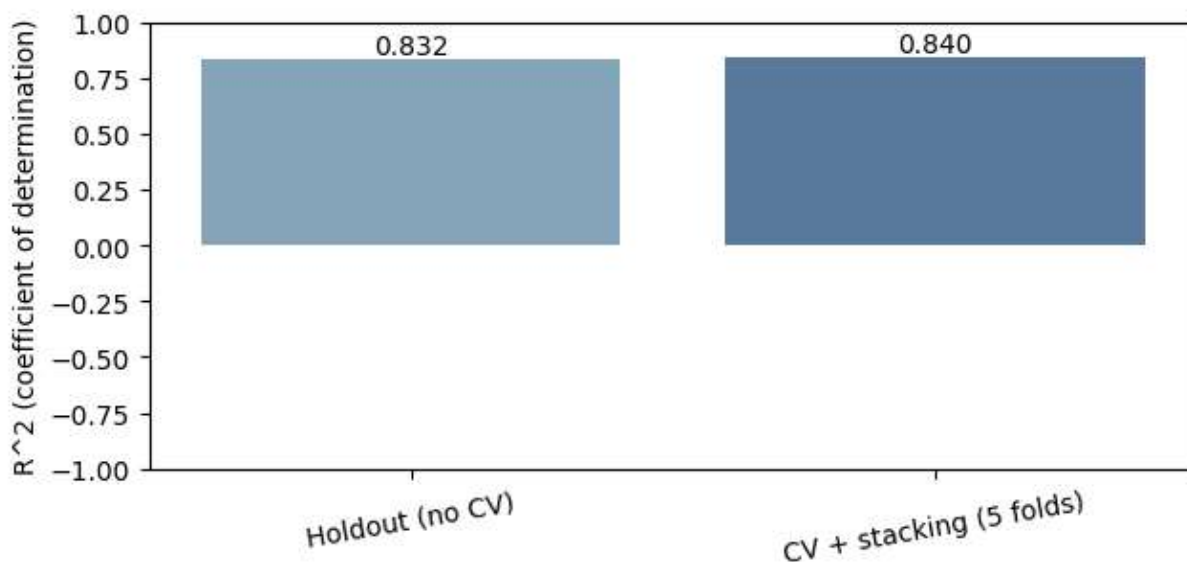
	approach	rmse	mae	r2
0	Holdout (no CV)	-332.873318	-264.649287	0.831856
1	CV + stacking (5 folds)	-324.792064	-263.167760	0.839921

```
In [14]: # Simple bar chart for R^2
plt.figure(figsize=(6.5,3.2))
ax = sns.barplot(data=cmp_df, x='approach', y='r2', hue='approach', palette=['#7aa6
ax.set_ylim(-1, 1)
ax.set_xlabel('')
ax.set_ylabel('R^2 (coefficient of determination)')
```

```

for p in ax.patches:
    height = p.get_height()
    ax.annotate(f'{height:.3f}', (p.get_x()+p.get_width()/2, height), ha='center',
plt.xticks(rotation=10)
plt.tight_layout()
plt.show()

```



Residual diagnostics

For regression, it's helpful to examine predictions vs actuals and residual distributions.

```

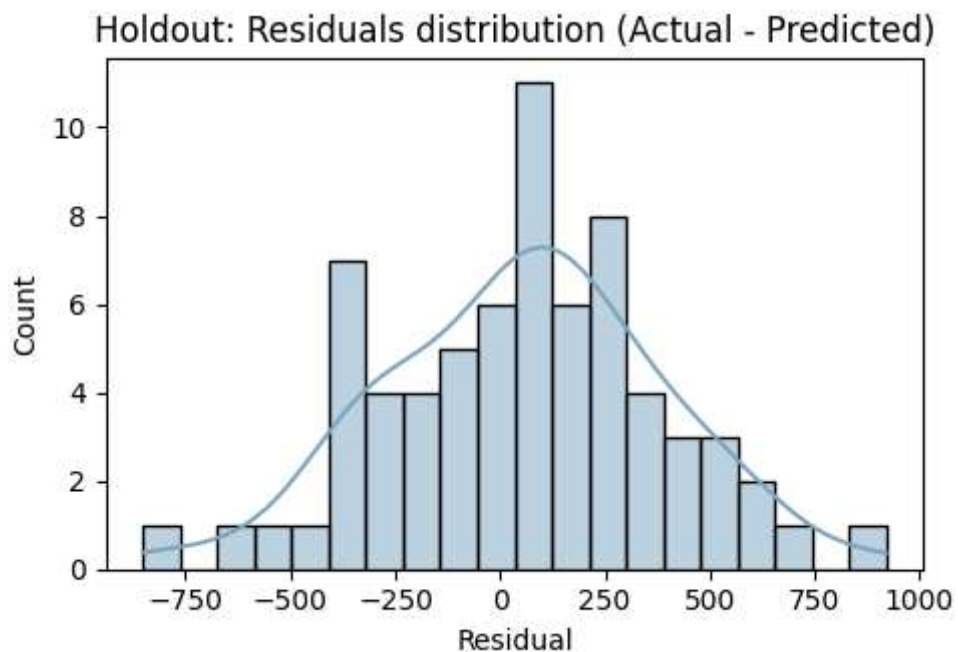
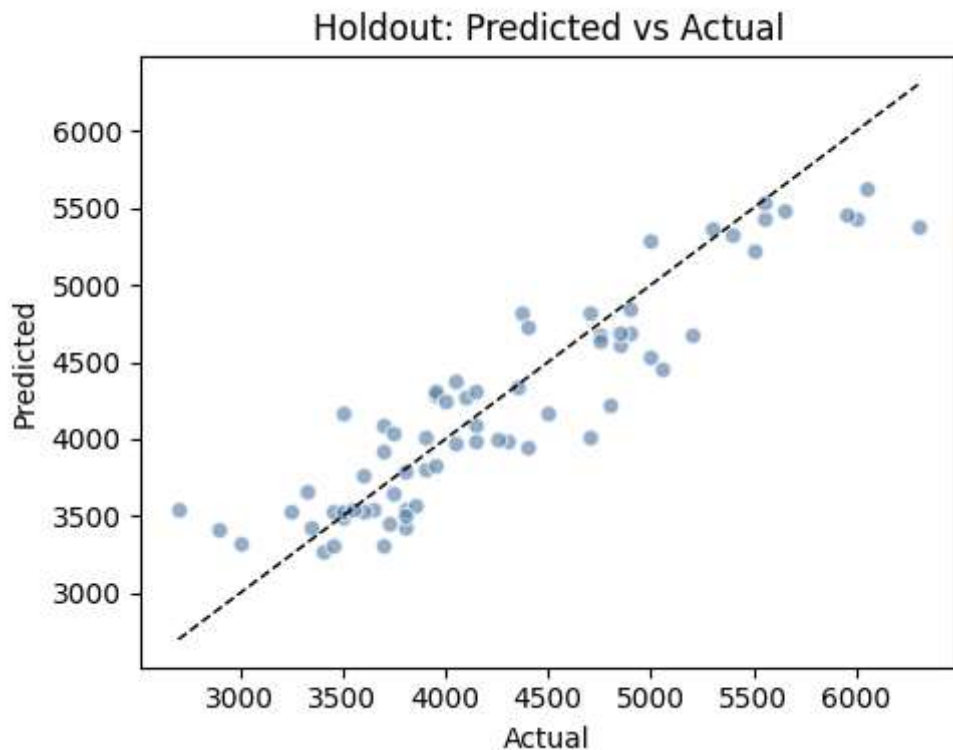
In [15]: def residual_plots(predictor, test_df, title_prefix='Model'):
    y_true = test_df[label].values
    y_pred = predictor.predict(test_df).values
    residuals = y_true - y_pred

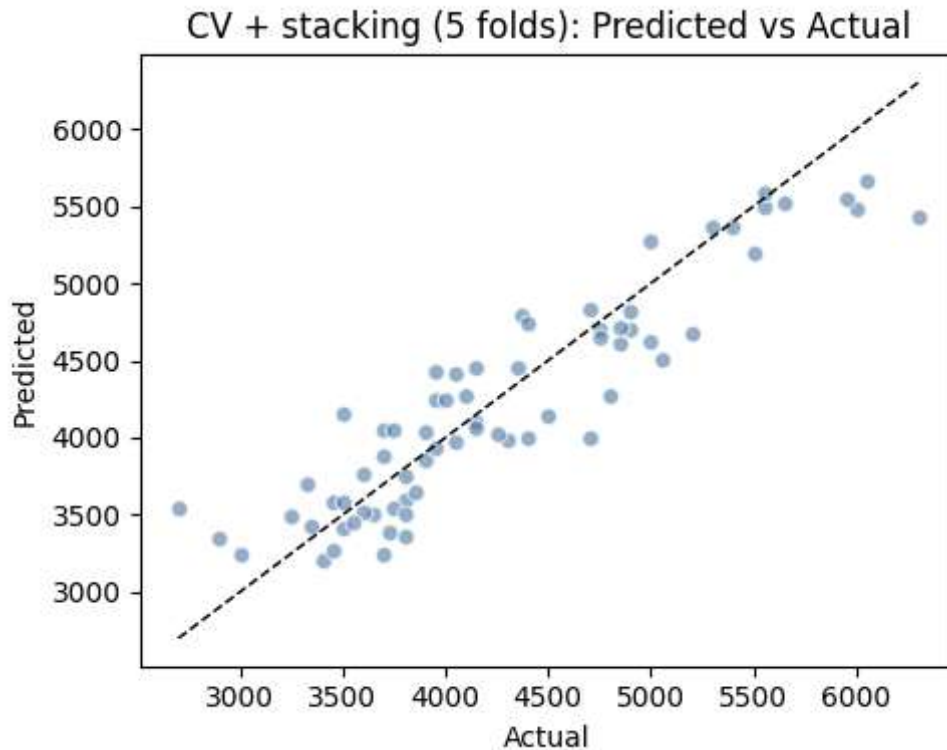
    # Predicted vs Actual
    plt.figure(figsize=(5,4))
    plt.scatter(y_true, y_pred, alpha=0.6, color='#4c78a8', edgecolor='white')
    minv, maxv = np.min([y_true.min(), y_pred.min()]), np.max([y_true.max(), y_pred.max()])
    plt.plot([minv, maxv], [minv, maxv], 'k--', lw=1)
    plt.title(f'{title_prefix}: Predicted vs Actual')
    plt.xlabel('Actual')
    plt.ylabel('Predicted')
    plt.tight_layout()
    plt.show()

    # Residuals histogram
    plt.figure(figsize=(5,3.5))
    sns.histplot(residuals, bins=20, kde=True, color='#7aa6c2')
    plt.title(f'{title_prefix}: Residuals distribution (Actual - Predicted)')
    plt.xlabel('Residual')
    plt.ylabel('Count')
    plt.tight_layout()
    plt.show()

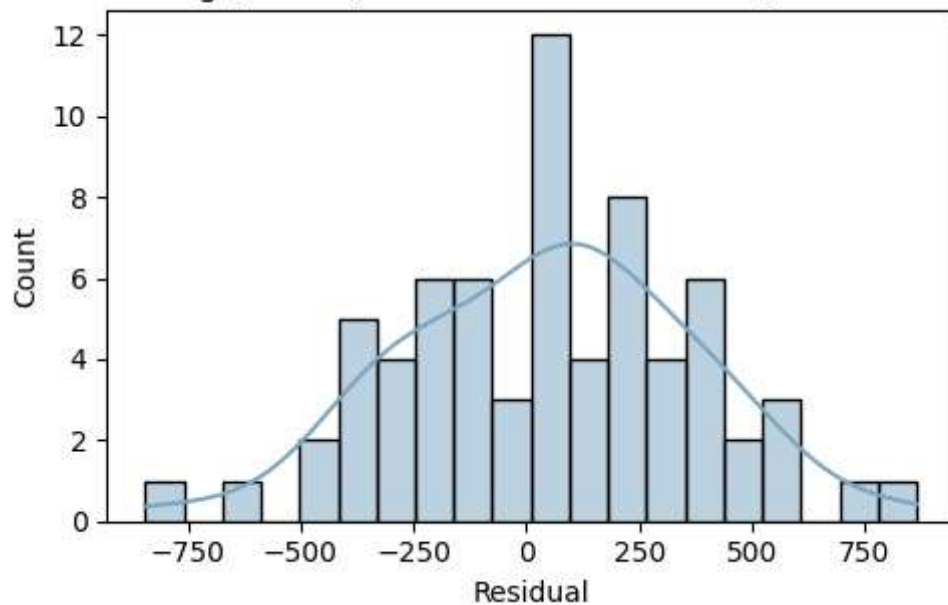
```

```
residual_plots(predictor_holdout, test_df, title_prefix='Holdout')  
residual_plots(predictor_cv, test_df, title_prefix='CV + stacking (5 folds)')
```





CV + stacking (5 folds): Residuals distribution (Actual - Predicted)



8) Takeaways and next steps

- For tabular regression, **CV + stacking/bagging** in AutoGluon is a strong default for stability and accuracy.
- Keep a **separate, untouched test set** for final grading.
- Increase `TIME_LIMIT_SEC` or use `presets='best_quality'` for higher accuracy in longer labs.

- Explore `predictor_cv.leaderboard(test_df)` to see which base learners drive performance.

Optional exercises

1. Change `num_bag_folds` to 3 or 10 and observe R^2 /RMSE changes.
2. Try `num_stack_levels=2` for deeper stacking.
3. Set `eval_metric='root_mean_squared_error'` and compare ranks vs `r2`.

In []: